A Reproducible Benchmark of Relational Database Access-Layer Performance in Express.js: A Configuration-Specific Comparison on PostgreSQL and MySQL

Mateusz Miotk[a,*]

[a]*Faculty of Mathematics, Physics and Informatics, University of Gdańsk, Wita Stwosza 57, 80-308, Gdańsk, Poland*

Abstract

Context: Node.js/Express services reach relational databases through a spectrum of access layers (native drivers, query builders, and object-relational mappers, or ORMs), yet practitioners choose between them largely on vendor benchmarks that are fragmented, PostgreSQL-only, and rarely report the tail (p99). **Objective:** We present a reproducible, byte-equivalence-checked benchmark harness and use it to compare eleven access-layer implementations under one clearly specified operating point on PostgreSQL and MySQL; the harness, not a generalized ranking, is the contribution. **Methods:** An identical Express application is served through nine documentation-selected access layers and two tuned native baselines, over five access patterns, on the latest stable library versions current at the July 2026 freeze. We report throughput with client-observed response-time p99 from 25 repeated runs per cell under three operating conditions: equal demand (a fixed 50-connection high-load point) across all patterns, and—on the deep fetch only—equal utilization (an open-loop sweep) and an exploratory equal compute budget. An automated cross-check verifies byte-identical responses before measurement. **Results:** The implementation-and-strategy difference concentrates in relation-materializing

*Corresponding author.
Email address: `mateusz.miotk@ug.edu.pl` (Mateusz Miotk)

patterns: the deep/nested fetch spreads 7.15× (PostgreSQL) and 4.96× (MySQL) between native and the slowest ORM, narrowing to 1.68× and 2.01× once every layer runs identical SQL, a compound control that bounds the combined query-strategy-and-formulation contribution without isolating a single mechanism. The read ordering is similar across engines (Spearman $\rho \geq 0.86$); aggregation and writes transfer only moderately ($\rho = 0.64$, 0.68). No layer exceeds $\approx 110\%$ of one core; the native driver leads all ten engine-pattern combinations and Prisma 7 sits mid-pack. An earlier version's five-core Prisma result, from a now-superseded generation, did not survive the re-run. **Conclusion:** Access-layer rankings can be version-sensitive, as one retired five-core result shows; the durable contributions are the reproducible, byte-equivalence-checked harness, its three-estimand design, and a checklist of genre-specific benchmarking pitfalls, released as a replication package.

Keywords: database access layer, object-relational mapping, Node.js, reproducible benchmarking, response-time tail (p99), empirical software engineering

1. Introduction

Express.js remains the most widely used web framework for Node.js, and almost every non-trivial service built on it eventually reaches a relational database through some *access layer*. The choice is not settled by the language or the framework: access layers trade raw performance for ergonomics, compile-time type safety, and protection from pitfalls such as the N+1 query problem, where a naive nested fetch silently issues one query per parent row instead of one for the whole result graph [1], a pervasive anti-pattern in real-world database-backed applications (Section 2). Native drivers expose the wire protocol directly (`pg` for PostgreSQL, `mysql2` for MySQL); query builders assemble SQL programmatically (Knex); and object-relational mappers (ORMs) map rows to objects and manage relationships automatically, following patterns such as Data Mapper and Active Record [2]. Node.js back-end performance under load is itself a recognized empirical topic [3, 4, 5], but that literature treats the runtime and web framework as the unit of comparison rather than varying the database access layer.

In practice, the most visible performance comparisons are *vendor benchmarks*: plentiful but narrow, published by the maintainers of the compared products, confined almost exclusively to PostgreSQL, and settling on a single metric family rather than reporting throughput and the tail together. In the sources searched, we did not identify one reporting the p99 tail behavior widely held to matter for user-perceived latency under load [6, 7]. The peer-reviewed literature is sparser still, limited to at most three ORMs, confined to PostgreSQL, and, where it benchmarks PostgreSQL against MySQL directly, holding the access layer fixed rather than varying it [8, 9, 10, 11, 12] (Section 2 surveys each).

In the sources we searched (Section 2) this leaves a gap along four axes (Table 1): no access-layer comparison we found includes MySQL, and no PostgreSQL/MySQL benchmark varies the access layer; no vendor-independent study spans the taxonomy broadly (the broad-coverage comparisons are vendor-authored, academic work covering at most three libraries [10, 11]); we did not identify one reporting throughput and the p99 tail jointly (the sole vendor suite stops at a single P95 at one load point [13]); and none measures client-observed performance through an HTTP framework, academic studies instead instrumenting query execution inside the process [11, 14].

This paper addresses that gap with a single, vendor-independent benchmark (authorship only; Section 3 defines the term): an *identical* Express application served through all nine access layers (two native drivers, Knex, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM) over five representative access patterns rather than a measured production workload (point read, keyset range scan, deep/nested fetch, per-author aggregation, and insert; Section 3). Seven layers run against *both* engines under an identical schema, dataset, and pool size; two tuned native-driver baselines mark what hand-tuned driver use achieves. Every (layer, engine, pattern) combination reports throughput (req/s) and client-observed response-time p99 at the fixed 50-connection high-load operating point jointly, from 25 repeated runs, with a utilization-controlled open-loop tail measured below saturation. The study separates two estimands: the *documentation-selected implementation-and-strategy* effect of each layer (RQ1–RQ3), and a controlled *same-SQL* contrast that bounds the residual difference once every layer executes identical SQL — changing query formulation, protocol, preparation, round-trips, and mapping together rather than isolating any one. The full harness (adapters, seed data, an autocannon-driven [15] load runner, and an automated cross-check that all layers return *byte-identical* responses on the four non-mutating patterns) is released as an open replication package.[1]

Three research questions structure the study, each scoped to the benchmarked implementations under the specified operating point:

RQ1 *(performance difference):* How large is the throughput and response-time-p99 *difference* of the benchmarked implementations relative to the native-driver baseline, and how does it vary across the five access patterns? We keep distinct three quantities that a fixed operating point conflates: *capacity* (saturating throughput, measured by the deep-fetch concurrency sweep), *high-load latency* (the p99 under equal external demand), and *latency at matched utilization* (each layer driven at equal fractions of its own capacity).

RQ2 *(engine dependence):* Does the observed layer ordering transfer across PostgreSQL and MySQL, or is it engine-dependent?

RQ3 *(consequential patterns):* For which access patterns is the choice of implementation most consequential in this configuration?

This paper makes four contributions; the **primary** ones are the harness and

[1] Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: `https://github.com/mmiotk/express-db-access-performance`; release v1.6.3 is permanently archived at Zenodo, `https://doi.org/10.5281/zenodo.21440942`.

the benchmark design, which outlast the pinned versions, while the third is a configuration-specific snapshot of a fast-moving ecosystem:

- An **open, reproducible harness**: a uniform adapter contract, a deterministic seed, a matrix runner with repeated randomized-block runs and paired request streams, a physical write-state reset, a same-SQL control with server-side protocol evidence, a coordinated-omission-corrected open-loop generator with a utilization-controlled mode, and an automated **correctness cross-check** asserting **byte-identical** responses across layers (Section 3).
- A **vendor-independent, broad-coverage benchmark design** spanning the taxonomy's three tiers (native driver → query builder → six ORMs), **dual-engine**, measured at the **Express level**, reporting throughput and response-time p99 **jointly**, a combination not identified in our documented search.
- **Configuration-specific observations** on the current versions and this host: no layer's throughput scales with additional application cores, the native driver leads all ten engine-pattern combinations with the difference concentrating in relation-materializing patterns, the read ordering is similar across engines while aggregation and writes transfer only moderately (visible only in a dual-engine design), and direct evidence that such rankings can be **version-sensitive** — one earlier headline retired by a re-run on newer versions (Section 4).
- A practitioner-oriented **checklist of four benchmarking pitfalls specific to access layers** (Section 5), several caught by the correctness cross-check and the rest by performance analysis.

2. Related Work

We organize prior work along four axes on which our study differs from it — taxonomy coverage, engine coverage, joint throughput/tail reporting, and vendor independence — summarized in Table 1. We located it with a structured scoping search (not a systematic review) of the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar (July 2026), crossing an access-layer set (*ORM*, "object-relational mapping", "query builder", "database access layer", and the seven library names) with Node.js/Express/ JavaScript, engine, and measurement terms over 2006–July 2026, followed by backward and forward citation chaining. We retained empirical comparisons of relational access layers or engines and excluded non-empirical, non-relational, and single-library studies; the exact strings, per-source dates, and screening decisions are archived with the replication package (`notes/related-work-search.md`). Because this is a scoping rather than exhaustive search, we scope every resulting gap claim to the sources searched.

2.1. Object-relational mapping and the access-layer spectrum

Access layers bridge the object-relational impedance mismatch between the relational model and the object graphs an application manipulates [16], spanning native drivers, query builders, and ORMs following patterns such as Data Mapper and Active Record [2] (Section 1). Torres et al. [17] survey two decades of ORM patterns; the trade-off they document — developer productivity against a hard-to-predict runtime cost — is the one this paper quantifies. The same question recurs in the polyglot-persistence debate over non-relational stores [18], a choice we hold fixed while varying only the access layer.

2.2. ORM performance and data-access anti-patterns

A parallel line studies ORM performance from the opposite direction, detecting and repairing the inefficient SQL that mapping frameworks emit rather than benchmarking layers. The costliest pattern is the N+1 query problem (Section 1); it and related redundant-query anti-patterns are pervasive across real-world web applications [19, 20, 21], and their performance impact has been quantified in ORM-backed systems [22, 1] and shown often *removable* by query synthesis, round-trip batching, or caching [23, 24, 25] (the JavaScript analogue being mis-sequenced `async`/`await` serializing independent I/O [26]). Closest to our design, Lorenz et al. [27] show the best O/R mapping strategy depends on the database technology, van Zyl et al. [28] that ORM performance is highly tuning-sensitive, and a recent study adds an energy dimension [29]. This literature establishes *where* ORM overhead concentrates (relation-materializing, N+1-prone access) and that it is configurable rather than fixed, which our per-pattern results confirm; it does not compare drivers, query builders, and ORMs head-to-head under controlled load, our aim here.

2.3. Peer-reviewed access-layer comparisons

The peer-reviewed literature that measures access layers head-to-head is narrow. Colley et al. [8] give the earliest systematic cross-language account, benchmarking eight ORM frameworks across Java, C#, Python, and PHP against a common schema, but *deliberately excludes JavaScript*, leaving Node.js, and Express in particular, without an academic baseline for the better part of a decade. Three recent studies partially address this: one, in *Procedia Computer Science*, compares Prisma against optimized raw SQL over eight query patterns on a containerized PostgreSQL setup [9] (cited for its methodology; differing library versions, hardware, workload, and harness mean we neither build on nor dispute its absolute speedup figures); a second compares Prisma, TypeORM, and Sequelize across four relation types, also on PostgreSQL, by response time, memory, and CPU [10]; and the most recent instruments the same three ORMs on PostgreSQL inside a NestJS service, timing internal query stages rather than client-observed requests [11], independently observing the concurrency-dependent Prisma ranking flip we also report (worst at single queries, best under parallel load). A further Express-based study optimizes a single stack (pooling, batching, caching) rather than comparing access layers [14].

2.4. Database-engine, SQL/NoSQL, and Node.js performance

A second, largely disjoint line benchmarks the layers immediately below and above the access layer without varying it directly. Salunke and Ouda [12] compare PostgreSQL and MySQL head-to-head under standard OLTP workloads (the only dual-engine precedent found), but benchmark the *engines themselves*, bypassing any application-level access layer, so their work cannot say how a driver, query builder, or ORM would change it; work contrasting SQL and NoSQL databases [30] likewise measures the store, not the access layer. On the Node.js side, Chaniotis et al. [3] established Node's viability as a web platform, Kuffel and Walter [4] tracked how its back-end performance drifts under sustained load, and do Carmo et al. [5] compared back-end frameworks more broadly. None isolates the database access layer as an independent variable, let alone varies it across the taxonomy studied here.

2.5. Standard benchmarks and benchmarking methodology

Standard benchmarks define how database systems are measured: YCSB [31] for cloud-serving stores and the TPC-C/TPC-E suites for OLTP. These target the *engine* and its transaction mix, saying nothing about the driver, query builder, or ORM this study isolates, but they set the methodological bar for controlled, repeatable load. A separate methodological literature explains why the vendor benchmarks' single-metric habit is itself a problem: Fruth et al. [7] and Raasveldt et al. [32] catalog database-benchmarking pitfalls (coordinated omission, warm-up, tool-induced distortion) that silently corrupt reported percentiles; Dean and Barroso [6] show why the tail, not the mean, determines user-perceived latency once a request fans out, as a deep/nested fetch does; and Li et al. [33] trace its hardware, OS, and application sources. Load-testing surveys report such measurement is often ad hoc [34, 35] and systems results frequently hard to reproduce [36], motivating a shared, reusable harness built for relevance, repeatability, and fairness [37, 38]. From this literature we take a design choice absent from the access-layer benchmarks surveyed above: report throughput *and* tail latency (p99) for the same run, rather than a single metric family chosen after the fact.

2.6. Practitioner and vendor benchmarks

A larger body of evidence comes from practitioners and vendors, each authored by a party with a stake in one of the compared products. Drizzle publishes both a Northwind micro-suite spanning nine targets (`pg`, Knex, Kysely, MikroORM, TypeORM, Prisma, and Drizzle itself, pooled and unpooled) and an official k6-driven load test reporting requests per second, average latency, P95, and CPU [13]. Prisma's own site compares Prisma, Drizzle, and TypeORM by median query latency over 500 iterations, reporting neither throughput nor any latency percentile [39]. IMDBench, maintained by Gel Data (formerly EdgeDB), adds Sequelize and node-postgres and reports both throughput and latency on three CRUD-style operations [40]. Together these three cover more of the taxonomy than the entire peer-reviewed literature combined, yet share the defects noted in Section 1: vendor authorship, PostgreSQL-only coverage, and a single reported metric family.

2.7. Positioning

In the sources searched through July 2026 (protocol archived with the replication package), no study combined broad taxonomy coverage, both engines, joint throughput/tail reporting, and vendor independence; each covers at most two or three of these four properties (Table 1). This study combines all four (Section 1), closing the same engine gap separating access-layer studies from engine-only work such as [12].

Table 1: Prior work on relational-database access-layer performance, positioned along the four axes on which this study differs from it. "Layers covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer.

Study	Layers covered	Engines	Metrics	Independent?
Colley et al. [8]	8 ORMs, 4 languages (no JS)	not stated	query latency	Yes
Procedia CS [9][†]	Prisma vs. raw SQL	PostgreSQL	latency, CPU	Yes
JCCT [10]	3 ORMs	PostgreSQL	latency, memory, CPU	Yes
JCSI [11]	3 ORMs	PostgreSQL	per-stage latency	Yes
Salunke and Ouda [12]	none (engine only)	PostgreSQL + MySQL	throughput, latency	Yes
Drizzle [13]	9 (broad)	PostgreSQL	throughput, latency, P95	No (vendor)
Prisma [39]	3 ORMs	PostgreSQL	median latency only	No (vendor)
IMDBench [40]	4 ORMs + driver	PostgreSQL	throughput, latency	No (vendor)
This study	**9 documentation-selected (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + response-time p99**	**Yes**

[†]Cited for its methodology only; its absolute speedup figures are not comparable to ours given the differing versions, hardware, workload, and harness.

3. Study Design

Our goal, in Goal–Question–Metric terms, is to *analyze* the relational database access layer of an Express.js service *for the purpose of* quantifying its performance cost *with respect to* throughput and response-time p99 *from the point of view of* an application developer choosing a layer *in the context of* PostgreSQL and MySQL back ends [41]. The design follows established guidelines for controlled experimentation in software engineering [42, 43], crossing seven portable access layers with both engines and all five patterns, plus two engine-specific native drivers and their tuned counterparts `pg-tuned` and `mysql2-tuned` (eleven layers: nine documentation-selected, two tuned reference baselines; full rationale in `METHODOLOGY.md`). The comparison is read at three levels, kept separate throughout. The **primary comparison** (RQ1–RQ3, Section 4) is each layer's *documentation-selected implementation and query/loading strategy* — the API each library's documentation presents first, not a claim about the most common or most performant one. A **same-SQL (raw-path) control** then *bounds* what remains once every layer executes identical SQL through its raw facility. The **mechanisms** that differ within a layer — eager-loading strategy, wire protocol, round-trips, statement preparation, hydration — change together and are *not* separately or causally identified; the same-SQL contrast is a bound, not a decomposition.

A fixed concurrency imposes equal *demand* on layers of unequal throughput and therefore drives them to unequal *utilization*, so RQ1 is characterized under three operating conditions rather than one privileged point. *Equal external demand* is the fixed 50-connection *high-load* operating point — the full five-pattern, two-engine matrix; high-load rather than saturated, because the throughput knee is measured only for the deep fetch (Supplement Figure S2), while 50 connections against a ten-connection pool keep about forty requests queued regardless of pattern (Controls, below). The other two conditions are a *limited* set restricted to the deep fetch, the pattern on which each layer's capacity was actually measured (Supplement Table S32 tabulates every experiment's scope): *equal utilization* is the coordinated-omission-corrected open-loop sweep, each layer offered a fixed fraction (50/70/85/95%) of its *own* saturating throughput, and an *equal compute budget* is the exploratory equal-CPU check, the server confined to a fixed number of cores. These answer different questions and need not agree, so we report all three and state which each result speaks to.

3.1. Factors and treatments

We vary three factors. The *access layer* has eleven levels spanning the taxonomy: the native drivers `pg` and `mysql2`, their tuned counterparts `pg-tuned` (named prepared statements reused across calls) and `mysql2-tuned` (binary protocol with server-side prepared statements via `execute()`), included as engine-specific tuned reference baselines rather than documentation-selected treatments; the query builder Knex; and the ORMs Drizzle (a lightweight, query-builder-style ORM), Prisma, Sequelize, TypeORM, Objection, and MikroORM. We classify a library by its dominant interface, not its self-description (Section 1); Objection, for instance, layers an ORM over Knex. The nine documentation-selected layers are a representative cross-section of the three tiers, not an exhaustive catalogue — each maintained, used, and, for the portable tiers, running against both engines — which excludes PostgreSQL-only Slonik and pg-promise as non-portable and Kysely as a query builder already covered by Knex (`METHODOLOGY.md` gives the full criteria). Vendor independence means authorship only: no evaluated library's maintainers were involved, though the consequential author choices (selection, schema, pool size, tuning) remain and are disclosed. The *engine* has two levels, PostgreSQL 18.4 and MySQL 9.7.1, and the *access pattern* has five, realized as HTTP endpoints (Table 2). The four native and tuned baselines are engine-specific (`pg`/`pg-tuned` on PostgreSQL, `mysql2`/`mysql2-tuned` on MySQL); the other seven layers run on both, giving $4 + 7 \times 2 = 18$ layer $\times$ engine cells and $18 \times 5 = 90$ measured configurations. Supplement Table S29 records which factors are held constant, vary, or are uncontrolled on this single virtualized host — the reason we report results for this configuration rather than as general library performance.

Table 2: The five access patterns and what each stresses.

Endpoint	Pattern	Stresses
`GET /posts/:id`	point read (PK)	per-query overhead
`GET /posts?before=`	keyset range scan	index seek + hydration
`GET /posts/:id/thread`	deep/nested fetch	join strategy, N+1 avoidance
`GET /authors/:id/summary`	aggregation	grouped counts / raw SQL
`POST /posts`	insert	write path

3.2. Objects: schema, workload, and data

The workload is a minimal blog domain (`authors` $1{-}*$ `posts` $1{-}*$ `comments` $*-1$ `authors`), exercising a one-to-many relationship and a two-hop join for the deep fetch. The five endpoints implement five representative access patterns [31] (Table 2): the keyset page uses `WHERE id < cursor ORDER BY id DESC LIMIT` n against the primary-key index rather than a large `OFFSET`; the deep/nested fetch returns a post with its author and every comment, each with its own comment-author; and the aggregation reports post count, comment count, and total views per author. The database is loaded from a deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) by the native driver, independently of any layer under test, so every engine receives the same seed *data payload* (byte-identical row values); on-disk representation, index layout, and collation naturally differ between engines. The working set fits in memory, so the measurements emphasize the access layer's per-request instruction, protocol, and mapping cost with disk-read latency largely removed, while engine-side execution, buffer management, locking, and logging remain part of every measurement [44]. Every request draws its target identifier uniformly at random from the full seeded range (posts 1–100,000, authors 1–2,000), so load spreads across the working set rather than one hot record; a skewed, production-like key distribution is a planned variant (Section 6).

3.3. The adapter contract and N+1 control

Every access layer implements the same factory interface (five query methods plus a teardown), so the Express server and load runner are layer-agnostic and each returns an *identical result shape*. Each uses that layer's *documentation-selected* mechanism to avoid the N+1 query problem on the deep fetch — a hand-written join for the native drivers, Knex, and the query-builder-style Drizzle (the tuned baselines reuse the native join), and relation eager loading (`include`, `withGraphFetched`, `populate`, or `relations`) for the data-mapper ORMs. The comparison is therefore one of each layer's *documentation-selected configuration* under a fixed adapter-selection rule, not a fixed query plan: a measured difference reflects the SQL each layer generates, its round-trip count, and how it materializes results, with one asymmetry inherent to any driver-versus-ORM comparison (the native default is hand-authored SQL, the ORM default the library's relation loader). Server-side statement logging captures the exact SQL, round-trip count, and `EXPLAIN` plans (released with the package); the per-endpoint counts differ by design (Supplement Table S2: the deep fetch takes one query in Sequelize and MikroORM, two in the native drivers, Knex, Drizzle, and TypeORM, and four in Prisma and Objection). The documentation-selected API was fixed by a rule decided *before* measurement — the eager-loading API each layer's official documentation presents first for the pinned version — a reproducible selection heuristic, not evidence that the chosen API is performance-optimal or the most common production choice, so we report a documentation-selected implementation-and-strategy estimand; Supplement Table S16 and `METHODOLOGY.md` record, per layer, that API with its documentation page and version, its round-trip count, the documented alternative we did *not* use, and that query logging, hooks, and validation are off.

To bound the combined contribution of query strategy and formulation, every adapter also implements a *same-SQL control*: identical two-statement deep-fetch SQL (and, per engine, identical `EXPLAIN` plans) with identical row mapping, executed through the layer's raw-SQL facility, mirroring the aggregation control (Section 4). Server-side capture, synchronized to a request marker so connection handshakes are excluded, confirms the two control statements are byte-identical across every layer on both engines; what differs is the wire protocol each driver negotiates (per-layer protocols in Supplement Table S14). The six ORMs split both ways on the join-versus-select-in default; a loading-strategy sensitivity check (Section 4) switches join-default data-mapper ORMs to select-in wherever the alternative is a byte-identical drop-in. To ensure no adapter silently returns less or issues N+1 queries, an automated cross-check (`bench/verify.mjs`) funnels every adapter's rows through shared canonical constructors and asserts full JSON byte equality against the native-driver baseline, on both engines, across 12 probes per adapter spanning the four non-mutating patterns and the same-SQL control, before any timing is collected. This check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; see Section 5).

3.4. Controls

Four controls isolate the access layer as the only variable. *(i) Pool size* is fixed at ten connections for every adapter, so the comparison reflects the access-layer choice, not pool tuning [45, 46] (for Prisma, whose default pool is core-count-derived, this is pinned via `connection_limit`). Because the load generator opens 50 concurrent connections against this ten-connection pool, roughly forty in-flight requests are always waiting in each library's acquisition queue; this queueing is deliberately part of what we measure and is held identical across layers (a 200 ms pool sampler confirmed the native PostgreSQL cell ran with its pool fully occupied and about 32 requests pending). *(ii) Identical query semantics*: each endpoint issues the same logical query across layers, aggregation using a single correlated-subquery formulation everywhere (the documented raw-SQL path), so the residual reflects the layer on an identical plan, not the generated SQL. *(iii) Write isolation*: because the insert endpoint mutates the dataset, every cell resets it before warm-up and before every measured run, and the write endpoint is measured in its own dedicated boot after the database's physical state is rebuilt (PostgreSQL: recreate from a seeded template; MySQL:

rebuild with `OPTIMIZE TABLE` and re-pin `AUTO_INCREMENT`), so physical layout and purge state cannot drift across replicates or leak between adapters. *(iv) Observer separation*: the HTTP load generator runs in a process separate from the server.

3.5. Measurement procedure

Load was generated with `autocannon` [15], a closed-loop (constant-concurrency) HTTP/1.1 generator [47] issuing requests at fixed concurrency and recording a latency histogram. Each of the 90 configurations was measured as 25 *repeated runs*: for each replicate we booted a fresh server process, opened 50 concurrent connections, ran a fifteen-second warm-up whose measurements were discarded — long enough for every layer to reach and sustain at least 95% of steady-state throughput, since managed runtimes do not always stabilize promptly [48] — then took one twelve-second measured run. Booting a fresh process per replicate prevents persistent application-process state (JIT, pool, heap) from carrying across replicates; all runs share one host and one short campaign, so replicate index and measurement order are treated as blocking factors, and cell order was reshuffled independently within each replicate (a forward-versus-reversed order check found no history effect on the read cells; supplement, *Measurement details*). Within a replicate, every layer and engine is driven by the identical request-id sequence from a PRNG seeded on the (endpoint, replicate) pair, so this paired-stream design removes between-layer sampling noise as a confound. The experimental unit is the freshly booted server run for one (layer, engine, endpoint) cell in one replicate; individual HTTP requests are subsamples. We report the median of the 25 replicate values of throughput and of the run-level percentiles p50, p90, p97.5, and p99; a 60 s re-measurement of the deep fetch preserves the p99 ranking exactly and barely moves each cell's p99 (Supplement Table S23), so the 12 s window is adequate for the tail. During measured runs we sampled server CPU and peak resident memory over the full process tree, with database and load-generator CPU sampled separately (supplement, *Measurement details*). Inserts were measured under each engine's *default* durability configuration (PostgreSQL `fsync` and `synchronous_commit` on; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`), the regime a deployment runs unless an operator relaxes it; a labelled secondary run relaxed all of those settings to isolate the durability mechanism (Supplement Table S3). To characterize behavior under load we swept the deep/nested fetch across 1–256 connections for every layer and engine [49], with an exploratory equal-CPU-budget check (Section 4); and a native `undici`-based constant-arrival generator drove the deep fetch under a fixed offered rate (250–4,000 req/s) rather than fixed concurrency, measuring latency from each request's intended send time and clipping timed-out requests at their cutoff, a coordinated-omission-corrected design [50, 51] that independently validates the closed-loop tail (Section 4).

3.6. Analysis

We separate *primary* outcomes, which carry the paired inference and the research-question claims, from *secondary* and *exploratory* outcomes, reported descriptively as controls and sensitivity analyses (Table 3): the primary outcomes are throughput and the closed-loop p99 on the five patterns against both engines at the fixed operating point.

Because absolute throughput is hardware-specific, we analyze *relative* differences within an engine; a pattern's *spread* is the documentation-selected native driver's median throughput divided by that of the slowest layer on that pattern (native-relative, so comparable across patterns). We report medians over the 25 repeated runs, the coefficient of variation as a stability signal [52, 53], and a percentile bootstrap 95% CI for the median from a fixed seed; these intervals capture within-campaign, run-to-run variability on this host and configuration, not variation across machines, versions, or deployments (Section 6). The design is a randomized block — within each replicate every layer runs the identical request stream — so adjacently ranked layers are compared with *paired* tests on their per-replicate throughput ratios (a two-sided sign-flip permutation test cross-checked with the Wilcoxon signed-rank test), reporting the

Table 3: Outcomes and their analysis role. The primary outcomes carry the paired inference and the research-question claims; the secondary and exploratory outcomes are reported descriptively as controls and sensitivity analyses that bound or contextualize the primary results, and are not treated as additional confirmatory hypotheses.

Role	Outcome	Analysis
Primary	Throughput (req/s), 5 patterns × 2 engines, documentation-selected layers, 50-connection operating point	Paired permutation, Wilcoxon, bootstrap median CI; spread & rank
Primary	Closed-loop p99 tail latency, same cells	Paired permutation, Wilcoxon, per-cell bootstrap CI on p99
Secondary	Same-SQL (raw-path) contrast	Bounds the residual (compound) same-SQL difference
Secondary	Sub-saturation open-loop tail (coordinated-omission corrected)	Lower-load latency companion to the high-load primary p99
Secondary	Layer×engine interaction (per pattern)	PG÷MySQL effect ratios (S28), blocked permutation (F on D)
Secondary	Combined app+db CPU efficiency, equal-CPU control	End-to-end compute view; operational confirmation of the CPU trade
Explor.	Aggregation fan-out; OFFSET vs. keyset; pool-size sweep; durability & isolation sensitivity; RSS	Descriptive; pitfall illustration and robustness checks

geometric-mean paired ratio, its bootstrap CI, and the paired dominance, plus a ±5%-margin TOST for the closest pair; we foreground these effect sizes and interval estimates over p-values, as recommended for randomized performance experiments [54, 55]. Because the seven adjacent pairs follow the *observed* ranking, their significances are a post-hoc, descriptive robustness check, not a confirmatory family. Adjacent-layer p99 gaps of one or two milliseconds sit at the millisecond measurement floor and are not read as real; conclusions rest on the large-ratio patterns, chiefly the deep fetch. The full construction — the permutation and bootstrap procedure and seed, tie handling, the layer×engine interaction test, the rank correlations, and the equivalence-margin rationale — is given in the supplement's *Statistical methods* section; the paired p99 comparisons are Supplement Table S30.

3.7. Experimental setup

The measurements were taken on 2026-07-16 to 2026-07-18 on a single host: the primary matrix ran overnight and the secondary experiments (orderinvariance, durability, same-SQL, open-loop, fan-out, equal-CPU, concurrency, utilization, pool-size, cluster, mixed-workload, wait-event, post-restart, and the longer-window tail) over the same window — a virtualized Intel Xeon Gold 6140 instance (16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.18.0; Express 5), with PostgreSQL 18.4 and MySQL 9.7.1 running locally. Each library is pinned to the *latest stable release compatible with the harness adapter contract* as of the freeze date (2026-07-15), recorded from the committed lockfile and an automated environment capture (Supplement Table S11); only Sequelize invokes the earlier-release exception (its version-7 line is a prerelease, so the current 6.x stable line is used). Because access-layer performance is version-sensitive, this pins a dated snapshot rather than a permanent ranking (Section 4).

3.8. Replication package

The harness — the Express application, the adapter contract, all nine documentation-selected adapters and the two tuned baselines, the deterministic seed, the `autocannon` matrix runner, the correctness cross-check, and the scripts regenerating every table in Section 4 — is released so the full matrix can be re-run with one command against a containerized or local PostgreSQL and MySQL.

4. Results

The tables report throughput (requests per second, higher is better) and tail latency (p99, milliseconds, lower is better) per access layer and engine. The two consequential patterns are in the main text (the deep/nested fetch, Table 4, and the insert, Table 5); the point read, keyset range scan, and aggregation are in Supplement Tables S12, S13, and S15. All numbers come from the run of Section 3. Each native driver is reported both as documentation-selected (`pg`, `mysql2`) and tuned (`pg-tuned`, `mysql2-tuned`), the tuned rows marking the throughput ceiling hand-tuned driver use reaches, not a level every documentation-selected layer attains without code changes. We compare *relative* differences within an engine (Section 3); two estimands recur below: the *documentation-selected implementation-and-strategy effect* (RQ1–RQ3) and the *same-SQL effect* (Supplement Table S14).

Table 4: Deep/nested fetch: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	3687 [3602–3737]	20 [18–21]	–	–
mysql2	native-driver	–	–	2382 [2357–2451]	28 [28–29]
pg-tuned	native-tuned	3746 [3684–3828]	18 [17–18]	–	–
mysql2-tuned	native-tuned	–	–	2415 [2387–2427]	25 [24–26]
knex	query-builder	2487 [2465–2505]	27 [26–28]	2007 [1989–2031]	30 [29–31]
drizzle	orm-lightweight	1865 [1837–1878]	35 [32–35]	1318 [1300–1336]	47 [46–48]
prisma	orm	1186 [1160–1203]	51 [51–53]	634 [618–644]	93 [89–95]
sequelize	orm	991 [977–998]	60 [59–61]	886 [868–898]	67 [66–68]
typeorm	orm	1204 [1188–1223]	50 [49–52]	1017 [987–1029]	57 [56–59]
objection	orm	1028 [1020–1037]	57 [56–58]	834 [828–855]	69 [67–71]
mikroorm	orm	516 [499–525]	116 [113–119]	480 [474–488]	120 [118–128]

4.1. RQ1: performance difference relative to the native baseline

The native driver leads all ten engine-pattern combinations: on PostgreSQL `pg` (or its tuned counterpart) is fastest on all five patterns, and `mysql2` leads the two reads, the deep fetch, aggregation, and the insert on MySQL. The current-generation ORMs sit mid-to-low throughout, Prisma among the lowest: seventh of eight on both point reads, near the bottom of aggregation on both engines, and last on the MySQL insert (Tables 4, 5; Supplement Tables S12 and S15). This retires an earlier version's headline of Prisma tying the native driver at a large CPU premium (Section 5).

`pg-tuned` and `mysql2-tuned` track their documentation-selected counter-

Table 5: Insert: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6402 [6242–6547]	11 [10–15]	–	–
mysql2	native-driver	–	–	1753 [1717–1763]	113 [109–119]
pg-tuned	native-tuned	6413 [6073–6571]	11 [10–12]	–	–
mysql2-tuned	native-tuned	–	–	1750 [1482–1759]	120 [111–130]
knex	query-builder	4841 [4629–4908]	15 [14–16]	1670 [1470–1692]	127 [117–137]
drizzle	orm-lightweight	4085 [3928–4146]	15 [14–18]	1730 [1324–1745]	120 [109–138]
prisma	orm	2774 [2750–2815]	23 [22–24]	886 [828–903]	153 [137–160]
sequelize	orm	2732 [2719–2747]	23 [21–25]	1478 [1375–1684]	125 [118–132]
typeorm	orm	2648 [2630–2694]	23 [22–26]	1345 [1301–1359]	125 [119–129]
objection	orm	3907 [3746–3980]	17 [15–23]	1728 [1598–1746]	117 [108–132]
mikroorm	orm	1979 [1949–1997]	30 [29–31]	1257 [1240–1268]	117 [109–125]

parts within a few percent on the single-round-trip patterns and edge them only on the deep fetch, where issuing more than one round trip lets reusable prepared statements and MySQL's binary protocol pay off (`pg-tuned` 3,746 vs. `pg` 3,687; `mysql2-tuned` 2,415 vs. `mysql2` 2,382); on PostgreSQL aggregation `pg-tuned` adds about 8% (7,538 vs. 6,978).

On the **point read**, the native-relative spread is 2.78× on PostgreSQL and 2.11× on MySQL (Supplement Table S12), with Prisma next-to-last, ahead of only MikroORM.

On the **deep/nested fetch** (a post, its author, and all comments with their authors), the native-relative spread is the widest measured: 7.15× on PostgreSQL (`pg` 3,687, 95% CI [3,602–3,737], down to MikroORM's 516 [499–525]) and 4.96× on MySQL (`mysql2` 2,382 [2,357–2,451] down to MikroORM's 480 [474–488]). The native driver leads outright on both engines — no tie at the top — and the data-mapper ORMs, especially MikroORM, trail furthest (Table 4). The comparison is paired (Section 3); every adjacent step of the

MySQL ranking is large relative to its bootstrap interval (Table 6).

Aggregation is the least layer-sensitive pattern: the native-relative spread is only 1.83× on PostgreSQL and 1.72× on MySQL (Supplement Table S15), and the native driver leads both. The documentation-selected difference is small when a layer forwards a single query and large only when it must materialize a nested object graph application-side, as on the deep fetch.

4.2. Same-SQL control

The *same-SQL control* (Section 3) bounds the residual difference once each layer runs the same statement rather than choosing its own query strategy: server-side capture confirms the statement text is byte-identical across layers on both engines while the wire protocols differ per layer, so this control is *same SQL, protocol disclosed*, not *same plan*. The contrast is composite — it changes query strategy, query count, prepared-statement behavior, the raw versus ORM API, and result marshalling together — so the residual difference is bounded but no single factor is isolated.

On the identical SQL, the deep-fetch native-relative spread collapses from 7.15× (PostgreSQL) and 4.96× (MySQL) to 1.68× and 2.01× (Supplement Table S14), so layers that trail on the documentation-selected fetch largely converge once they run the same statement — bounding the query-strategy-and-formulation contribution without isolating a single mechanism. Prisma is the slowest layer on the raw path itself; we conjecture this reflects dispatch and marshalling overhead in its raw API rather than SQL execution, a hypothesis the composite contrast cannot confirm. A loading-strategy sensitivity check (Supplement Table S18) rules out a mere default-strategy artifact: switching the join-default ORMs (Sequelize, MikroORM) to select-in makes them slower, whereas Objection's batched-select-in default is about 1.6× slower than a single join on MySQL, so part of *its* deficit there is a strategy choice rather than a fixed cost.

4.3. RQ2: engine dependence of the ranking

The relative ordering is largely, but not uniformly, stable across engines. The Spearman rank correlation between the PostgreSQL and MySQL orderings of the seven portable layers (descriptive; Section 3) is high on the reads — $\rho = 0.86$ on the point read, 0.89 on the range scan, and 0.86 on the deep fetch — but only *moderate* on aggregation ($\rho = 0.64$) and the insert ($\rho = 0.68$), which transfer less reliably though neither is uncorrelated. Prisma is the notable mover: mid-pack on PostgreSQL inserts (2,774 req/s, fourth of the seven portable layers) but last on MySQL inserts (886, behind even MikroORM's 1,257) — a rank flip invisible to a single-engine benchmark (Supplement Table S27). The cost's *magnitude* also depends on the engine (Supplement Table S28, per-layer PostgreSQL÷MySQL ratios). Every layer is faster on PostgreSQL, so the layer×engine interaction is in the *spread* of that advantage, not its sign: across the three reads it stays within ≈1.0–1.9×, but the insert scatters from 1.6× (MikroORM) to 3.2× (Prisma), reordering the layers across engines. The blocked permutation test (Section 3) only confirms this interaction is nonzero; its floor p says nothing about size. Single-engine results are thus portable in order for reads but not in degree, most loosely for writes.

On the **keyset range scan** the native driver again leads (native-relative spread 4.21× on PostgreSQL, 3.85× on MySQL; Supplement Table S13), driven disproportionately by MikroORM alone (905 / 856 req/s); the earlier collapse under `OFFSET` pagination was a workload artifact, not an engine or layer property (Section 5).

Inserts are the pattern where the engine dominates most visibly, under each engine's *default* durability configuration (Section 3). On MySQL the faster layers cluster within about 5% near an engine-imposed floor (`mysql2` 1,753 down to Knex 1,670 req/s; Table 5), with Prisma the exception at 886 (last), widening the nominal MySQL spread to 1.98×. Direct `performance_schema` instrumentation attributes the floor to the commit path: relaxing durability lifts the flush-bound native driver and query builder about 2× on the MySQL insert while the slower ORMs, limited by their own write overhead, gain less (Supplement Tables S3, S19), so the engine's commit path is the floor for the lean layers, not a layer property. PostgreSQL, by contrast, spreads 3.23× (`pg` 6,402 down to MikroORM 1,979) and is only weakly durability-sensitive (≤14%), so there the access layer, not the engine, remains the larger factor.

4.4. RQ3: which patterns matter most

Ranking the five patterns by native-relative spread gives, on PostgreSQL, deep fetch (7.15×) > range scan (4.21×) > insert (3.23×) > point read (2.78×) > aggregation (1.83×); MySQL keeps the same head and tail (deep fetch 4.96×, aggregation 1.72× last) while the insert and point read swap in the middle, the insert compressing because MySQL's engine floor, not the access layer, sets its ceiling (RQ2). The deep/nested fetch is thus the most consequential pattern on both engines and aggregation the least. The primary deep-fetch workload uses posts with roughly ten comments; a fan-out sweep from 0 to 500 comments shows the native-relative spread is a roughly fixed multiplier with graph size rather than growing with it (exploratory; an earlier three-run sweep suggesting a growing spread did not survive replication).

4.5. Tail latency

We report the closed-loop p99 for every cell at the 50-connection operating point, so the tail includes connection-acquisition queueing — the tail a deployment sees under load. At this high-load point tail latency tracks throughput: on the deep/nested fetch, PostgreSQL's p99 ladder runs from `pg` 20 to MikroORM 116 ms (Table 4), monotonic below the native driver — a penalty throughput-only vendor benchmarks miss — and a paired p99 analysis (Supplement Table S30) resolves every adjacent pair on both engines except two near-ties, otherwise mirroring the throughput ranking. Driving the deep fetch under the coordinated-omission-corrected open-loop harness (Supplement Tables S6, S17) reproduces this ordering: below saturation the corrected tails are flat and small, and beyond each layer's capacity the tail collapses rather than degrading gracefully, the data-mapper ORMs earliest. Offering each layer

50/70/85/95% of *its own* saturating throughput (Supplement Tables S21–S22), the tails converge (near 2–5 ms at 50% utilization on both engines), so the large high-load gap (pg 20 ms versus MikroORM 116 ms) largely dissolves at matched utilization: it is a capacity-and-queueing effect, not a difference in tail latency at comparable utilization.

4.6. Measurement stability and effect sizes

Across the 25 repeated runs the coefficient of variation of deep-fetch throughput is at most 4.0% on either engine (Supplement Tables S1, S9), well below the between-layer spreads; dispersion is larger only on the insert, where the MySQL cells are noisiest (up to 15.9%, bimodal under group-commit batching; raw distributions in Supplement Figure S1), so the insert rank correlation is reported conservatively. Every measured request succeeded (zero errors or non-2xx across all 90 cells); those counters are load-bearing, since a broken cell returning errors faster than real work can spuriously lead a ranking (Section 5), and a post-restart re-measurement reproduced the ranking with absolute throughput up to 16% lower (Supplement Table S20). The effect sizes carry the ordering (Table 6): six of the seven adjacent MySQL deep-fetch steps are geometric-mean ratios ≥ 1.15 with the faster layer ahead in every replicate, well above the $\leq 4.0\%$ CV, while the narrowest (Sequelize over Objection, 1.05) sits at the $\pm 5\%$ practical-equivalence margin; the permutation and Wilcoxon tests agree (Bonferroni intact) but descriptively, as a post-hoc check on the deep fetch, not independent confirmation.

4.7. Scaling with concurrency

A concurrency sweep from 1 to 256 connections (Supplement Figure S2) shows the three-band ordering (native driver, then Knex and Drizzle, then the data-mapper ORMs) emerges by 8 connections and holds at every operating point above about 32; at a single connection the layers are far closer together (about 320–1,020 req/s) and Prisma is mid-pack throughout, never approaching the native driver. The 50-connection operating point sits above the knee for every layer on the deep fetch, so the RQ1 ranking is not an artifact of a single point.

4.8. Resource footprint

On the current library generation the application-side CPU cost is uniform (Supplement Tables S5, S10): on the deep/nested fetch every layer draws about one core (100–110% of one logical core, the maximum MikroORM's 110%), so no layer buys throughput with extra application cores. What differs is database-side load: the native `pg` driver runs at roughly 100% application CPU but issues lean SQL that drives the database harder ($\approx$ 355% database CPU on the PostgreSQL deep fetch), whereas the ORMs sit at the same application CPU and drive lower database CPU. The native driver therefore leads on throughput and end-to-end compute efficiency at once — counting combined application-plus-database CPU per request, `mysql2` completes 1,254 req per combined-CPU-second on the MySQL deep fetch against Prisma's 384 and MikroORM's 356 (Supplement Table S5) — so the slower layers are also less compute-efficient. An exploratory equal-CPU check confirms no layer converts extra application cores into throughput; a layer's low per-process throughput is instead recoverable by horizontal scaling (Supplement Tables S4, S24). Memory differences are smaller (Supplement Table S10): the query builder and lightest ORMs are cheapest (Knex 215, Objection 221, TypeORM 225 MB peak RSS), while Drizzle, Prisma, and MikroORM use more (297–356 MB), which governs how many replicas a given load requires [56].

5. Discussion

Access-layer rankings can be version-sensitive; the instrument is the contribution. An earlier campaign found Prisma (then 5.22, on an in-process Rust query engine) tying the native driver on the deep fetch while drawing about five application cores; re-running the *identical* harness on the latest stable releases at the freeze did not reproduce it — no layer now draws more than about one core, and

Table 6: Paired comparison of adjacently ranked access layers on the deep/nested fetch (MySQL), over the 25 repeated runs. Because every layer runs the identical per-replicate request stream, layers are compared on their per-replicate throughput ratios: median throughput (req/s; the per-cell bootstrap CI is in the pattern tables), the geometric-mean paired ratio A/B with a paired bootstrap 95% CI, paired dominance (fraction of replicates in which A exceeds B), and the two-sided paired sign-flip permutation p (20000 permutations; a value of $5.0{\times}10^{-5}$ is the resolution floor $1/(B{+}1) = 1/20{,}001$, i.e. the observed statistic exceeded every permutation; the Wilcoxon signed-rank test agrees, replication package).

Comparison (A > B)	med. A	med. B	ratio A/B [95% CI]	dom.	perm. p
mysql2 > knex	2382	2007	1.19 [1.18–1.20]	1.00	$5.0{\times}10^{-5}$
knex > drizzle	2007	1318	1.53 [1.51–1.55]	1.00	$5.0{\times}10^{-5}$
drizzle > typeorm	1318	1017	1.30 [1.28–1.32]	1.00	$5.0{\times}10^{-5}$
typeorm > sequelize	1017	886	1.15 [1.13–1.16]	1.00	$5.0{\times}10^{-5}$
sequelize > objection	886	834	1.05 [1.03–1.07]	0.88	0.0001
objection > prisma	834	634	1.33 [1.31–1.36]	1.00	$5.0{\times}10^{-5}$
prisma > mikroorm	634	480	1.31 [1.29–1.33]	1.00	$5.0{\times}10^{-5}$

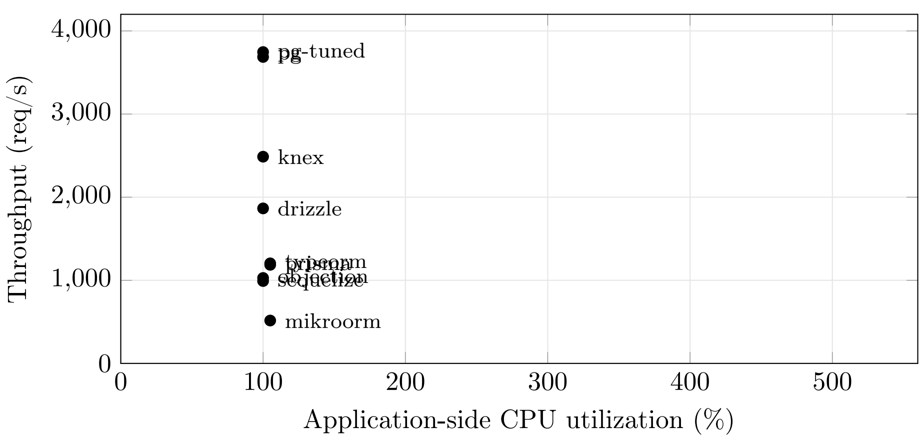

Figure 1: Throughput versus application-side CPU on the deep/nested fetch (PostgreSQL, ten-connection pool, medians of 25 replicates). Every layer sits at about 100–110% of one application core; the native driver leads on the throughput axis, and the slower ORMs are low on throughput at the same CPU. CPU is the server process only; database-side CPU is excluded for all layers alike.

Prisma has fallen to mid-to-low throughput (Section 4). The non-reproduction is a fact; its cause is a hypothesis (only Prisma changed architecture, so its Rust-free rewrite is the likeliest driver, but the re-freeze moved the whole toolchain at once). The headline is therefore version-sensitive [28, 57], and the durable contributions are the reusable harness, the benchmark design, and the pitfalls checklist.

Implications for practice. RQ1 and RQ3 agree that the documentation-selected implementation-and-strategy difference is specific to the access *pattern*, concentrating where a layer must hydrate rows into objects and resolve associations rather than merely issue SQL (the object-relational impedance mismatch [16] made concrete): the native-relative spread is widest on the deep/nested fetch and narrowest on aggregation. Round-trip count alone does not set throughput — on the PostgreSQL deep fetch the single-query MikroORM is the slowest of all while the two-query Drizzle outruns the four-query Prisma (Supplement Table S2; Table 4) — so how a layer materializes a result graph matters more than how many statements it issues.

Compute cost and horizontal scaling. Under current versions the throughput differences are no longer bought with extra application cores: every layer sits at roughly one application core, and the native driver's lead comes from issuing lean SQL that pushes work onto the database tier (Section 4; Figure 1). A low per-process throughput is not a fixed ceiling — Prisma recovers native-like aggregate throughput under a shared-port `node` cluster at four workers, trading footprint and energy for throughput [29]. The open-loop cross-run further shows that a closed-loop benchmark alone would underestimate how abruptly, not just how much, a heavyweight ORM's tail degrades once offered load crosses its capacity.

Writes: a partly engine-bound cost under default durability. RQ2 has a clear answer on inserts, under each engine's *default* durability configuration: the native driver leads on both engines, PostgreSQL's insert throughput stays strongly layer-dependent even with fsync on (3.23×; Table 5), whereas on MySQL the faster layers cluster within about 5% near an engine-level commit floor (the per-commit redo-log-then-binlog double flush [44]; Supplement Table S19), and relaxing durability does not overturn this engine contrast. The write ranking transfers only moderately across engines (Spearman $\rho = 0.68$, against 0.86–0.89 for the reads), so a single-engine write comparison invites the wrong generalization either way; an engine-only benchmark such as [12] cannot expose this interaction, whereas the dual-engine design does, consistent with earlier work showing database technology modulates mapping-strategy cost [27]. A transactional multi-statement write (a post and five comments in one transaction) narrows the representative-layer spread to about $3.7\times$ (Supplement Table S8), below the widest read spread, because the single commit amortizes the flush across six inserts; Prisma, mid-pack on the single-row insert, falls to the bottom transactionally, so the transactional ordering does not simply track the single-row one.

Methodological lessons: a checklist for access-layer benchmarking. Reaching the figures above meant confronting four confounds that would otherwise have corrupted the comparison silently, which we distill into a reusable checklist (full detail in the supplement's *Benchmarking-pitfalls checklist*): an **aggregation fan-out** (a join evaluated before `SUM`, inflating the aggregate) and a driver result-shape mismatch, both *correctness* bugs the automated byte-equality cross-check caught; **OFFSET pagination** collapsing on MySQL, a pagination-strategy cost easily misread as an access-layer weakness; **write-workload contamination** from an unreset growing table; and a **query-formulation confound** where a mis-scoped pre-aggregation re-scanned the whole table, a query-plan artifact masquerading as an access-layer effect [58]. The write path needs its own gate: MikroORM 7's removal of `persistAndFlush` made every MikroORM insert throw a 500 that returned faster than a real insert, briefly *leading* the MySQL write ranking until the non-2xx counter flagged it, so the error counter is the write path's correctness gate as byte-equality is the read path's. None of the four is schema-specific; each is a generic failure mode extending the database-benchmarking pitfalls of Fruth et al. [7] and Raasveldt et al. [32] to the access-layer sub-genre, and at least one plausibly explains part of the gap between some vendor-reported and independently verified figures in this space.

Practical guidance. The practical synthesis follows from RQ1–RQ3 and applies to the benchmarked implementations in this configuration, workload, and host rather than to access-layer categories in general. The native driver leads throughput on every pattern on both engines, and the ORM penalty concentrates on relation-materializing traffic while read-simple patterns and aggregation spread far less. Where a service's hot path is dominated by deep or nested fetches the access layer is consequential and should be benchmarked for the specific application: the tested thin paths (native driver and Knex) led here *for throughput and response-time p99*, but whether that generalizes to other implementations, schemas, or workloads is a hypothesis for broader study, not a category law or a general default. Where the workload is read-simple or write-bound (especially on MySQL, whose commit floor dominates layer differences under default durability) the throughput cost is small, so a full ORM's productivity and type safety — not measured here — may reasonably decide the choice, and a layer's low per-process throughput is recoverable by horizontal scaling. These are performance findings only: the decision also turns on correctness, maintainability, security, and developer productivity, which this study does not measure and which may outweigh the differences reported here; the recommendations rest on the single host, default durability, and pinned versions of Section 6, and the same-SQL results bound the documentation-selected cost rather than licensing raw SQL in practice.

6. Threats to Validity

We organize the threats along the four standard validity categories [59, 42].

Construct validity. Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) at the Express endpoint, not query-execution time at the driver or database boundary — deliberate, since it captures the full request round trip a deployed service incurs (Express routing, JSON serialization), though the overhead we attribute to the *access layer* may partly reflect these surrounding costs. We hold them near-constant: every adapter satisfies a documented *adapter contract* and funnels its rows through shared canonical constructors (a fixed field set, key order, and value typing, under `TZ=UTC`), and the automated cross-check (`bench/verify.mjs`) asserted full JSON byte equality against the native-driver baseline, on both engines, across the four non-mutating patterns before any timing run (Section 3). During development that check caught a defect: PostgreSQL's schema declared `created_at` as a timezone-naive timestamp although the column is `timestamptz`, so Drizzle's PostgreSQL responses carried instants shifted by the host's UTC offset — invisible to throughput and to any coarser equivalence check; it was fixed (`withTimezone: true`) before the campaign reported here. The fixed-payload baseline floor sits well above the fastest measured cell (Section 3), so the framework floor compresses, rather than manufactures, between-layer differences, which are therefore attributable to how each layer reaches an identical response, not to what is returned. Writes are the one endpoint that mutates state; the reported finding uses each engine's *default*, vendor-standard durability configuration (Section 3), not one we relaxed ourselves, so the write spreads (Table 5) describe a standard deployment rather than a tuning choice; a symmetric relaxed-durability secondary confirms the cross-engine gap is not an artifact of either configuration (Section 5, Supplement Table S3).

Internal validity. A naively configured ORM can trigger N+1 queries and appear artificially slow for reasons unrelated to its inherent cost; every adapter must use its layer's documentation-selected eager-loading strategy on the deep-fetch endpoint (Section 3). The byte-level correctness cross-check above caught two defects during development, both fixed before the measurements reported here (the checklist in Section 5). Explicit warm-up phases per cell (Section 3), set to exceed the slowest layer's cold-start stabilization, absorb

JIT, pool-fill, and plan-cache effects, guarding against environmentally induced measurement bias [60]; two further controls rule out non-layer confounds — benchmark rows reset before every cell (physically rebuilt for writes), and one correlated-subquery plan for aggregation across layers (Section 3). Every layer and engine within a replicate is driven by the identical, PRNG-seeded id sequence, and a forward-versus-reversed cell-order check found no detectable history effect (Section 3), addressing the concern that processing order could confound layer identity with run position. A residual risk is coordinated omission in the load generator, which can under-report tail latency at saturation [7, 50]; a native, coordinated-omission-corrected constant-arrival cross-run on the deep fetch (Section 3) confirms the tail ranking holds under that stricter model (Section 4; Supplement Table S6).

External validity. All measurements come from a single schema, dataset size, and 16-vCPU virtualized host, with a connection pool fixed at ten, against specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1). The fixed pool makes this an *equal configured connection limit* comparison; a per-layer pool-size frontier (Supplement Tables S7, S25) shows the ordering preserved from a pool of 1 to 50, and a mixed read/write workload (Supplement Table S26) preserves the read ordering, so the fixed pool does not drive the ranking. Results may not transfer to larger working sets, other pool sizes, or other versions, and library performance ages quickly, so rather than freeze an arbitrary snapshot we pin the latest stable release compatible with the harness at the freeze date (Supplement Table S11); Prisma is measured at 7.8, the Rust-free client that replaced the earlier in-process Rust query engine [61]. Two version choices are disclosed rather than resolved: Sequelize runs on its stable 6.x line (7.x is prerelease), and Prisma's MySQL path uses the MariaDB driver adapter, since Prisma ships no `mysql2` adapter. What re-establishes a current ranking is the instrument, not the table, so only the *relative* ordering within an engine is intended to travel; absolute req/s reflect this substrate. The working set is memory-resident, a hot-cache regime that maximizes layer differences; as a workload becomes I/O-bound the spreads should compress toward 1×, and the fan-out sweep (Section 4) shows the small deep-fetch graph is representative of a 0–500-child range. Three same-host concerns are checked and found not to drive the ranking: a resource-isolation run pinning application, database, and generator to disjoint cores agrees with the shared-host medians within about 2% (ordering unchanged); a multi-worker check (Supplement Table S24) confirms the low-per-process layers scale out near-linearly, so single-process throughput is not the aggregate ceiling; and a ten-minute sustained-load check moves throughput by at most $\pm 1.4\%$, so the short runs are not sampling a transient. Longer-horizon drift [4] and a two-machine cross-run over a dedicated link remain planned.

Conclusion validity. Our analysis (Section 3) reports medians with the coefficient of variation and bootstrap 95% confidence intervals as stability signals and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios (paired sign-flip permutation, cross-checked with Wilcoxon; Table 6). Two bounds reinforce the ordering: the widest spreads dwarf the run-to-run CV (deep-fetch throughput varies by at most $\approx 4.0\%$ across the 25 repeated runs on either engine; Supplement Tables S1, S9), and the ordering holds across the concurrency sweep at ≥ 32 connections (Supplement Figure S2). Rather than assert a specific statistical power, which would need a prospective effect-size and variance model we did not prespecify, we rest the conclusions on effect sizes and interval estimates: the adjacent-rank pairs combine large paired ratios — down to 1.05 for the closest, near the $\pm 5\%$ practical-equivalence margin — with non-overlapping bootstrap intervals above the run-to-run dispersion, the paired permutation test agreeing only as a descriptive post-hoc check (six pairs at the resolution floor, all seven surviving Bonferroni). Booting a fresh process per replicate prevents persistent process state from carrying across replicates [62], but the replicates share one host and one campaign, so we treat replicate index and measurement order as blocks rather than claiming full independence; more replicates or longer runs would tighten the intervals further.

7. Conclusion and Future Work

This paper replaces vendor-benchmark claims with a vendor-independent, reproducible, configuration-specific comparison of relational database access layers in Express.js, serving an identical application through a representative taxonomy (native drivers, a query builder, and six ORMs) across PostgreSQL and MySQL, over five access patterns, reporting throughput and response-time p99 together (Section 1).

The documentation-selected implementation-and-strategy difference is concentrated, not uniform: sharpest for the deep/nested fetch that assembles a nested object graph application-side, and modest for point reads and aggregation (Section 4). The same-SQL control narrows the spread substantially but, changing several mechanisms together, bounds rather than attributes the difference to any single one (Section 4). On the current library versions no layer's throughput scales with additional application cores and the native driver leads all ten engine-pattern combinations; the read ordering is similar across the two engines while aggregation and writes transfer only moderately, a caution against generalizing single-engine results (Section 5). That access-layer rankings can be version-sensitive is, here, a measured result, not a hypothesis: an earlier version's headline did not survive re-running the same harness on newer versions. Development also surfaced a reusable checklist of benchmarking pitfalls specific to this genre, one of them, a fast-but-wrong write path that passed the read-level cross-check, added by this very re-run (Section 5).

These findings held under a concurrency sweep (1 to 256 connections) and rest on large between-layer effect sizes with non-overlapping bootstrap intervals, the descriptive adjacent-rank permutation tests agreeing (Section 4). Further extensions would deepen the evidence — a two-machine cross-run to bound observer interference beyond the open-loop check, additional engines and dataset sizes, and a dedicated study of the N+1 penalty and cold-start latency — and the released harness, exercised again by this revision's re-run against Prisma 7, makes them straightforward, addressing the repeatability gap documented in computer-systems research [36].

CRediT authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The author is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Data availability

The complete replication package (benchmark harness, database schema and deterministic seed, all adapter implementations, the byte-level correctness cross-check, raw per-cell measurements, and the scripts that generate every table in this paper) is publicly available at `https://github.com/mmiotk/express-db-access-performance` and permanently archived at Zenodo as release v1.6.3 (`https://doi.org/10.5281/zenodo.21440942`). A reproducibility summary — versions, requirements, the run commands, the time and resources needed, and which results the harness regenerates automatically from the archived raw data — is in Supplement Table S31 and `REPRODUCE.md`.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used a large language model (Claude, Anthropic) to assist with drafting and editing the manuscript and with implementing and analyzing the benchmark harness. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article. All AI-assisted code was verified: the statistical estimators carry unit tests against hand-computed values and known closed-form properties (`bench/stats.test.mjs`, run by `npm test`), every reported table cell traces to a raw per-cell record through a committed generator (`MANIFEST.md`), and the byte-level correctness cross-check independently confirms that all layers return identical responses before any timing is trusted.

References

[1] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Detecting performance anti-patterns for applications developed using object-relational mapping, in: Proceedings of the 36th International Conference on Software Engineering (ICSE), ACM, 2014, pp. 1001–1012. doi:10.1145/2568225.2568259.

[2] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, Boston, MA, 2002.

[3] I. K. Chaniotis, K.-I. D. Kyriakou, N. D. Tselikas, Is Node.js a viable option for building modern web applications? a performance evaluation study, Computing 97 (10) (2015) 1023–1044. doi:10.1007/s00607-014-0394-9.

[4] P. Kuffeł, B. Walter, Changes in Node.js server-side application performance characteristics over time under load, in: Advances in Information Systems Development, Vol. 77 of Lecture Notes in Information Systems and Organisation, Springer, 2025, pp. 275–294. doi:10.1007/978-3-031-87880-0_14.

[5] K. X. Carmo, F. Ferreira, E. Figueiredo, Performance evaluation of back-end frameworks: A comparative study, in: Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI), ACM, 2024, pp. 1–9. doi:10.1145/3658271.3658314.

[6] J. Dean, L. A. Barroso, The tail at scale, Commun. ACM 56 (2) (2013) 74–80. doi:10.1145/2408776.2408794.

[7] M. Fruth, S. Scherzinger, W. Mauerer, R. Ramsauer, Tell-tale tail latencies: Pitfalls and perils in database benchmarking, in: Performance Evaluation and Benchmarking (TPCTC 2021), Vol. 13169 of Lecture Notes in Computer Science, Springer, 2022, pp. 119–134. doi:10.1007/978-3-030-94437-7_8.

[8] D. Colley, C. Stanier, M. Asaduzzaman, The impact of object-relational mapping frameworks on relational query performance, in: 2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE), IEEE, 2018, pp. 47–52, java, C#, Python, PHP — deliberately excludes JavaScript/Node. doi:10.1109/iccecome.2018.8659222.

[9] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, R. R. Siswanto, Optimizing database access strategy: A performance analysis comparison of raw sql and prisma orm, Procedia Comput. Sci. 269 (2025) 1201–1210. doi:10.1016/j.procs.2025.09.061.

[10] J. Attala, C. Khemapatpapan, Comparing the performance of prisma, typeorm, and sequelize on postgresql, J. Comput. Creat. Technol. 3 (2) (2025) 201–213. doi:10.14456/jcct.2025.16.
URL https://so13.tci-thaijo.org/index.php/jcct/article/view/2330

[11] S. Zhadko-Bazilevych, Analysis of ORM framework approaches for Node.js, J. Comput. Sci. Inst. 37 (2025) 426–430. doi:10.35784/jcsi.7951.

[12] S. V. Salunke, A. Ouda, A performance benchmark for the PostgreSQL and MySQL databases, Future Internet 16 (10) (2024) 382. doi:10.3390/fi16100382.

[13] Drizzle Team, Drizzle orm benchmarks, `https://orm.drizzle.team/benchmarks`, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026). (2024).

[14] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, M. A. Ibrahim, Optimizing asynchronous performance in Node.js with Express and PostgreSQL, Procedia Comput. Sci. 269 (2025) 172–181. doi:10.1016/j.procs.2025.08.270.

[15] M. Collina, et al., autocannon: fast http/1.1 benchmarking tool for node.js, `https://github.com/mcollina/autocannon`, version 7.15.0; accessed 10 July 2026. (2023).

[16] C. Ireland, D. Bowers, M. Newton, K. Waugh, A classification of object-relational impedance mismatch, in: 2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA), IEEE, 2009, pp. 36–43. doi:10.1109/DBKDA.2009.11.

[17] A. Torres, R. Galante, M. S. Pimenta, A. J. B. Martins, Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design, Inf. Softw. Technol. 82 (2017) 1–18. doi:10.1016/j.infsof.2016.09.009.

[18] P. J. Sadalage, M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, Addison-Wesley, Upper Saddle River, NJ, 2012.

[19] J. Yang, P. Subramaniam, S. Lu, C. Yan, A. Cheung, How not to structure your database-backed web applications: A study of performance bugs in the wild, in: Proceedings of the 40th International Conference on Software Engineering (ICSE), ACM, 2018, pp. 800–810. doi:10.1145/3180155.3180194.

[20] C. Yan, A. Cheung, J. Yang, S. Lu, Understanding database performance inefficiencies in real-world web applications, in: Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM), ACM, 2017, pp. 1299–1308. doi:10.1145/3132847.3132954.

[21] S. Shao, Z. Qiu, X. Yu, W. Yang, G. Jin, T. Xie, X. Wu, Database-access performance antipatterns in database-backed web applications, in: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2020, pp. 58–69. doi:10.1109/ICSME46990.2020.00016.

[22] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational mapping frameworks, IEEE Trans. Softw. Eng. 42 (12) (2016) 1148–1161. doi:10.1109/TSE.2016.2553039.

[23] A. Cheung, A. Solar-Lezama, S. Madden, Optimizing database-backed applications with query synthesis, in: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2013, pp. 3–14. doi:10.1145/2491956.2462180.

[24] A. Cheung, S. Madden, A. Solar-Lezama, Sloth: Being lazy is a virtue (when issuing database queries), in: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, ACM, 2014, pp. 931–942. doi:10.1145/2588555.2593672.

[25] T.-H. Chen, W. Shang, A. E. Hassan, M. Nasser, P. Flora, CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), ACM, 2016, pp. 666–677. doi:10.1145/2950290.2950303.

[26] A. Turcotte, M. D. Shah, M. W. Aldrich, F. Tip, DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript, in: Proceedings of the

44th International Conference on Software Engineering (ICSE), ACM, 2022, pp. 774–785. doi:10.1145/3510003.3510097.

[27] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, H. Plattner, Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies, in: Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS), 2017. doi:10.24251/HICSS.2017.592.

[28] P. van Zyl, D. G. Kourie, L. Coetzee, A. Boake, The influence of optimisations on the performance of an object relational mapping tool, in: Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT), ACM, 2009, pp. 150–159. doi:10.1145/1632149.1632169.

[29] A. Bonvoisin, C. Quinton, R. Rouvoy, Understanding the performance-energy tradeoffs of object-relational mapping frameworks, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2024, pp. 626–636. doi:10.1109/SANER60148.2024.00069.

[30] Y. Li, S. Manoharan, A performance comparison of SQL and NoSQL databases, in: 2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM), IEEE, 2013, pp. 15–19. doi:10.1109/PACRIM.2013.6625441.

[31] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC), ACM, 2010, pp. 143–154. doi:10.1145/1807128.1807152.

[32] M. Raasveldt, P. Holanda, T. Gubner, H. Mühleisen, Fair benchmarking considered difficult: Common pitfalls in database performance testing, in: Proceedings of the Workshop on Testing Database Systems (DBTest), ACM, 2018, pp. 1–6. doi:10.1145/3209950.3209955.

[33] J. Li, N. K. Sharma, D. R. K. Ports, S. D. Gribble, Tales of the tail: Hardware, OS, and application-level sources of tail latency, in: Proceedings of the ACM Symposium on Cloud Computing (SoCC), ACM, 2014, pp. 1–14. doi:10.1145/2670979.2670988.

[34] Z. M. Jiang, A. E. Hassan, A survey on load testing of large-scale software systems, IEEE Trans. Softw. Eng. 41 (11) (2015) 1091–1118. doi:10.1109/TSE.2015.2445340.

[35] P. Leitner, C.-P. Bezemer, An exploratory study of the state of practice of performance testing in Java-based open source projects, in: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE), ACM, 2017, pp. 373–384. doi:10.1145/3030207.3030213.

[36] C. Collberg, T. A. Proebsting, Repeatability in computer systems research, Commun. ACM 59 (3) (2016) 62–69. doi:10.1145/2812803.

[37] K. Huppler, The art of building a good benchmark, in: Performance Evaluation and Benchmarking (TPCTC 2009), Vol. 5895 of Lecture Notes in Computer Science, Springer, 2009, pp. 18–30. doi:10.1007/978-3-642-10424-4_3.

[38] S. E. Sim, S. Easterbrook, R. C. Holt, Using benchmarking to advance research: A challenge to software engineering, in: Proceedings of the 25th International Conference on Software Engineering (ICSE), IEEE, 2003, pp. 74–83. doi:10.1109/ICSE.2003.1201189.

[39] Prisma, Query performance benchmarks, https://benchmarks.prisma.io/, prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026). (2024).

[40] Gel Data (EdgeDB), Imdbench: Orm benchmarks, https://github.com/geldata/imdbench, accessed 10 July 2026. (2024).

[41] V. R. Basili, H. D. Rombach, The TAME project: Towards improvement-oriented software environments, IEEE Trans. Softw. Eng. 14 (6) (1988) 758–773. doi:10.1109/32.6156.

[42] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, 2nd Edition, Springer, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29044-2.

[43] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, IEEE Trans. Softw. Eng. 28 (8) (2002) 721–734. doi:10.1109/TSE.2002.1027796.

[44] S. Harizopoulos, D. J. Abadi, S. Madden, M. Stonebraker, OLTP through the looking glass, and what we found there, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, ACM, 2008, pp. 981–992. doi:10.1145/1376616.1376713.

[45] K. Gupta, M. Mathuria, Improving performance of web application approaches using connection pooling, in: 2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA), IEEE, 2017, pp. 355–358. doi:10.1109/ICECA.2017.8212833.

[46] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, M. Kalinowski, Data management in microservices: State of the practice, challenges, and research directions, Proc. VLDB Endow. 14 (13) (2021) 3348–3361. doi:10.14778/3484224.3484232.

[47] B. Schroeder, A. Wierman, M. Harchol-Balter, Open versus closed: A cautionary tale, in: Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2006. URL https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html

[48] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, L. Tratt, Virtual machine warmup blows hot and cold, Proceedings of the ACM on Programming Languages 1 (OOPSLA) (2017) 1–27. doi:10.1145/3133876.

[49] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, M. Stonebraker, Staring into the abyss: An evaluation of concurrency control with one thousand cores, Proc. VLDB Endow. 8 (3) (2014) 209–220. doi:10.14778/2735508.2735511.

[50] G. Tene, How NOT to measure latency, `https://www.infoq.com/presentations/latency-response-time/`, conference presentation; origin of the term "coordinated omission" (accessed 10 July 2026). (2015).

[51] B. Krishnamachari, How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results, `https://arxiv.org/abs/2605.00428` (2026).

[52] A. Georges, D. Buytaert, L. Eeckhout, Statistically rigorous Java performance evaluation, in: Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA), ACM, 2007, pp. 57–76. doi:10.1145/1297027.1297033.

[53] C. Laaber, J. Scheuner, P. Leitner, Software microbenchmarking in the cloud. how bad is it really?, Empir. Softw. Eng. 24 (4) (2019) 2469–2508. doi:10.1007/s10664-019-09681-1.

[54] A. Arcuri, L. Briand, A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering, Softw. Test. Verif. Reliab. 24 (3) (2014) 219–250. doi:10.1002/stvr.1486.

[55] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychol. Bull. 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.

[56] L. M. Vaquero, L. Rodero-Merino, R. Buyya, Dynamically scaling applications in the cloud, ACM SIGCOMM Comput. Commun. Rev. 41 (1) (2011) 45–52. doi:10.1145/1925861.1925869.

[57] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, A. Iosup, Methodological principles for reproducible performance evaluation in cloud computing, IEEE Trans. Softw. Eng. 47 (8) (2021) 1528–1543. doi:10.1109/TSE.2019.2927908.

[58] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proc. VLDB Endow. 9 (3) (2015) 204–215. doi:10.14778/2850583.2850594.

[59] T. D. Cook, D. T. Campbell, Quasi-Experimentation: Design and Analysis Issues for Field Settings, Houghton Mifflin, Boston, MA, 1979.

[60] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.

[61] Prisma, Prisma 7 performance and benchmarks, `https://www.prisma.io/blog/prisma-7-performance-benchmarks`, announces the query-engine rewrite from the Rust engine to a TypeScript client (accessed 13 July 2026). (2025).

[62] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 International Symposium on Memory Management (ISMM), ACM, 2013, pp. 63–74. doi:10.1145/2464157.2464160.